

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA0302 |

COURSE CODE: Reinforcement Learning

CO2: Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks. (BL2, BL3)

Industry Problem Task

NAME : SUGUNA G

REG NO : 192425032

SOLUTIONS

1.Explain how Reinforcement Learning can be applied to optimize delivery routes in a logistics system. Identify the possible states, actions, rewards, and policy. Discuss how continuous learning improves efficiency over time.

Explanation

Reinforcement Learning (RL) helps a logistics company find the best delivery routes by allowing an agent (delivery vehicle) to learn from experience. The agent interacts with the road network and receives rewards based on delivery performance.

RL Components

- **States:** Vehicle location, traffic condition, fuel level, remaining deliveries.
- **Actions:** Move to next road, choose alternative route, stop for refueling.
- **Rewards:**
 - +10 for successful delivery
 - +5 for shortest route
 - -5 for traffic delay
 - -10 for excess fuel consumption

- **Policy:** Select the route that maximizes total reward while minimizing delivery time.

Continuous Learning

- Learns from previous deliveries.
- Adapts to changing traffic conditions.
- Improves route selection over time.
- Reduces delivery cost and fuel usage.

Conclusion: RL continuously improves delivery efficiency by learning the best routing strategy.

CODE:

```
import random
```

```
states = ["Warehouse", "Road A", "Road B", "Customer"]
```

```
actions = ["Move", "Change Route"]
```

```
state = "Warehouse"
```

```
for i in range(5):
```

```
    action = random.choice(actions)
```

```
    if action == "Move":
```

```
        reward = 10
```

```
        state = random.choice(states)
```

else:

reward = -2

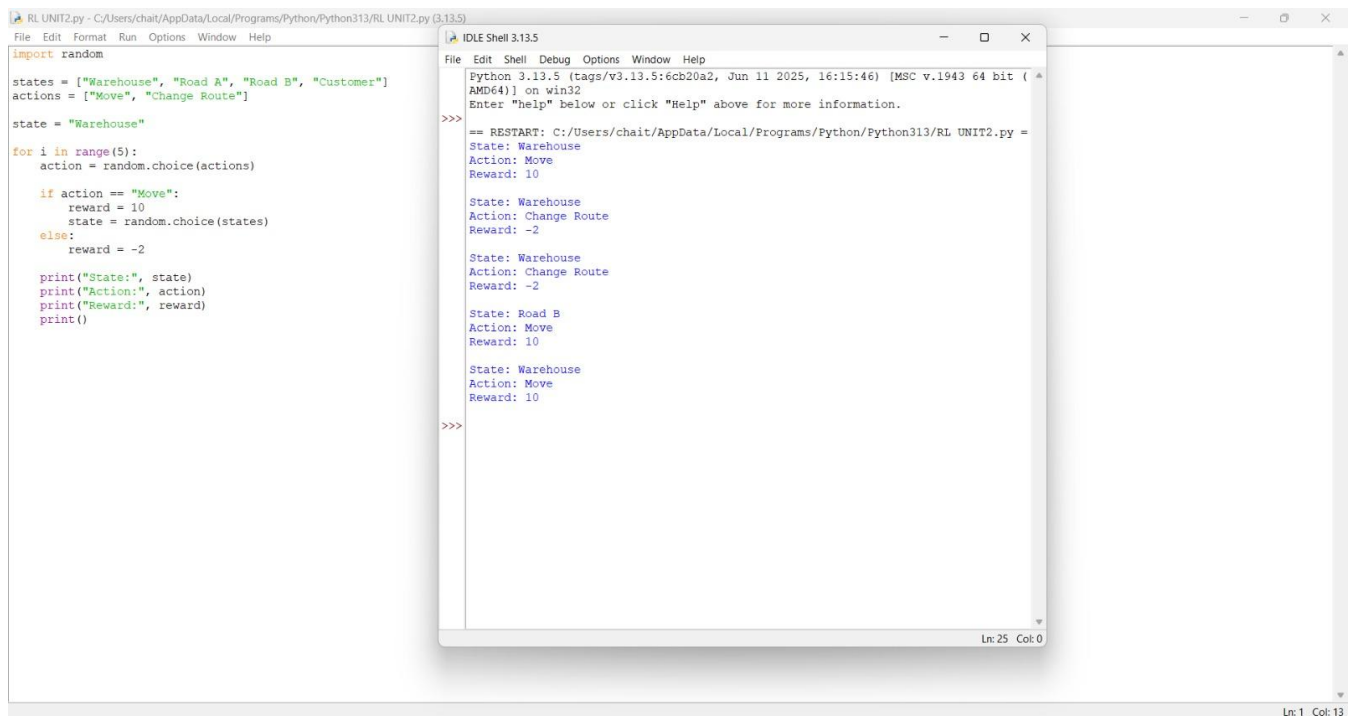
print("State:", state)

print("Action:", action)

print("Reward:", reward)

print()

OUTPUT:



The screenshot shows a Python IDE with two windows. The left window, titled 'RL UNIT2.py', contains the following code:

```
import random

states = ["Warehouse", "Road A", "Road B", "Customer"]
actions = ["Move", "Change Route"]

state = "Warehouse"

for i in range(5):
    action = random.choice(actions)

    if action == "Move":
        reward = 10
        state = random.choice(states)
    else:
        reward = -2

    print("State:", state)
    print("Action:", action)
    print("Reward:", reward)
    print()
```

The right window, titled 'IDLE Shell 3.13.5', shows the output of the code execution:

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
== RESTART: C:/Users/chaith/AppData/Local/Programs/Python/Python313/RL UNIT2.py ==
State: Warehouse
Action: Move
Reward: 10

State: Warehouse
Action: Change Route
Reward: -2

State: Warehouse
Action: Change Route
Reward: -2

State: Road B
Action: Move
Reward: 10

State: Warehouse
Action: Move
Reward: 10

>>>
```

The status bar at the bottom right of the IDE indicates 'Ln: 25 Col: 0'.

2. Apply Reinforcement Learning concepts to a fraud detection system in fintech. Define the state space, action space, and reward mechanism. Explain how exploration strategies improve detection accuracy.

Explanation

RL can detect fraudulent financial transactions by learning which transactions are genuine and which are suspicious.

RL Components

- **State Space:** Transaction amount, location, customer history, transaction time.
- **Action Space:**
 - Approve transaction
 - Reject transaction
 - Flag for manual review
- **Reward Function:**
 - +10 for correctly detecting fraud
 - +5 for approving genuine transactions
 - -20 for approving fraudulent transactions
 - -5 for false alarms

Exploration Strategy

- Tests different detection methods.
- Learns new fraud patterns.
- Improves accuracy against evolving fraud techniques.

Conclusion: RL increases fraud detection accuracy while reducing false positives.

CODE:

```
import random
```

```
transactions = ["Normal", "Fraud"]
```

```
for i in range(5):
```

```
    state = random.choice(transactions)
```

```
    action = random.choice(["Approve", "Reject"])
```

```
    if state == "Fraud" and action == "Reject":
```

```
        reward = 10
```

```
    elif state == "Normal" and action == "Approve":
```

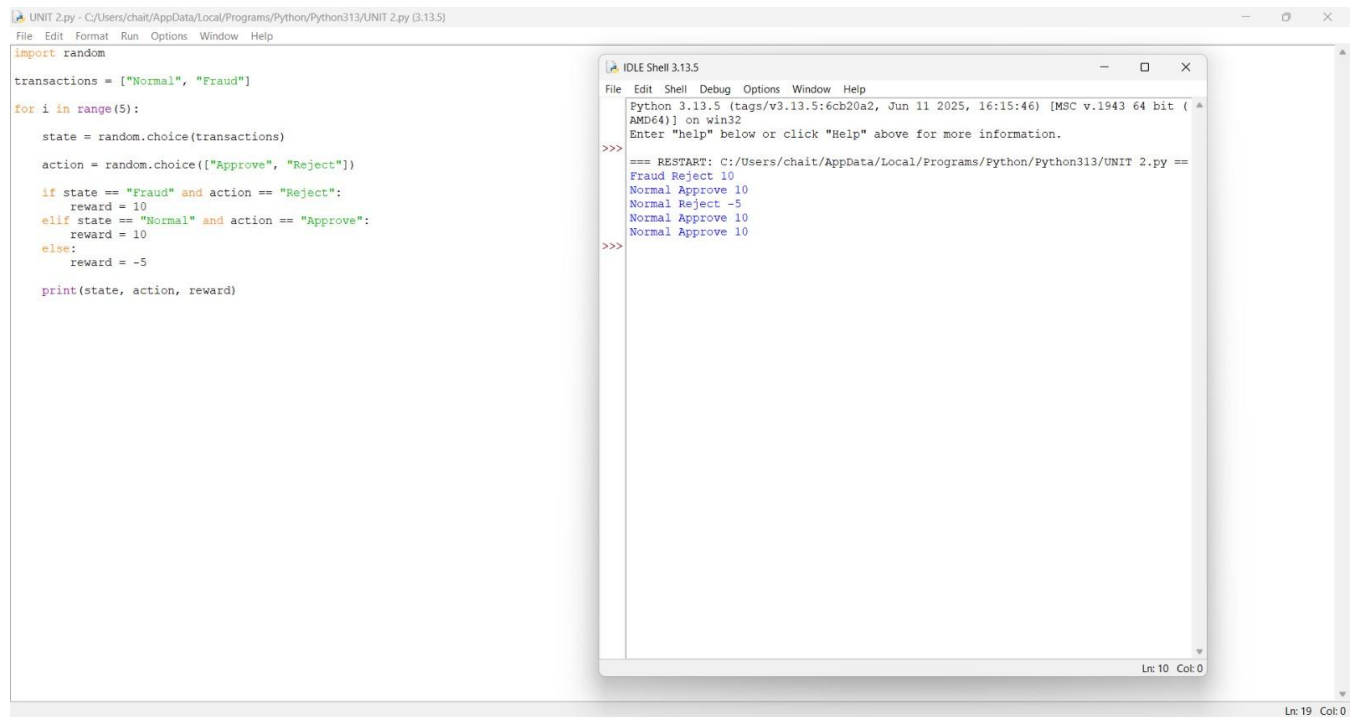
```
        reward = 10
```

```
    else:
```

```
        reward = -5
```

```
    print(state, action, reward)
```

OUTPUT:



```
UNIT 2.py - C:/Users/chaht/AppData/Local/Programs/Python/Python313/UNIT 2.py (3.13.5)
File Edit Format Run Options Window Help

import random

transactions = ["Normal", "Fraud"]

for i in range(5):
    state = random.choice(transactions)
    action = random.choice(["Approve", "Reject"])
    if state == "Fraud" and action == "Reject":
        reward = 10
    elif state == "Normal" and action == "Approve":
        reward = 10
    else:
        reward = -5
    print(state, action, reward)
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/chaht/AppData/Local/Programs/Python/Python313/UNIT 2.py ==
Fraud Reject 10
Normal Approve 10
Normal Reject -5
Normal Approve 10
Normal Approve 10
>>>
```

Ln: 10 Col: 0

Ln: 19 Col: 0

3. Illustrate how Reinforcement Learning is used in a streaming platform for content recommendation. Discuss the impact of delayed rewards, user feedback, and changing preferences on learning performance.

Explanation

Streaming services use RL to recommend movies or videos based on user preferences.

RL Components

- **States:** User watch history, likes, viewing time.
- **Actions:** Recommend different movies or shows.
- **Rewards:**
 - +10 if user watches completely
 - +5 if user likes content
 - -5 if recommendation is skipped

Learning Challenges

- **Delayed Rewards:** User satisfaction is known after watching.
- **User Feedback:** Likes, ratings, and watch time improve recommendations.
- **Changing Preferences:** RL adapts when user interests change.

Conclusion: RL provides personalized recommendations that improve user engagement.

CODE:

```
import random
```

```
import numpy as np
```

```
Q = np.zeros((2, 2))
```

```
alpha = 0.1
```

```
gamma = 0.9
```

```
for _ in range(1000):
```

```
    state = random.randint(0, 1)
```

```
    action = np.argmax(Q[state])
```

```
    reward = 10 if state == action else -5
```

```
    next_state = random.randint(0, 1)
```

```
    Q[state][action] += alpha * (
```

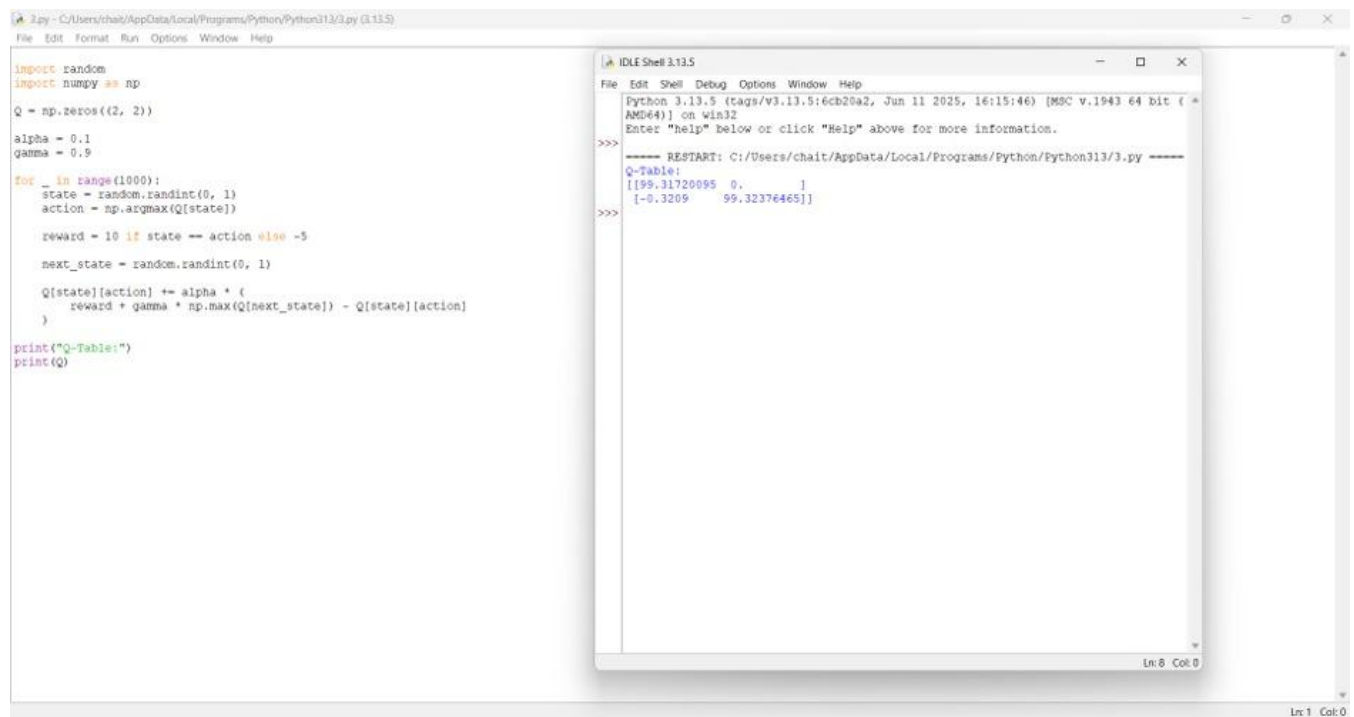
```
        reward + gamma * np.max(Q[next_state]) - Q[state][action]
```

```
)
```

```
print("Q-Table:")
```

```
print(Q)
```


OUTPUT:



The image shows a screenshot of a Python script being executed in an IDE. The script is a Q-learning implementation. The output window shows the execution progress and the final Q-table.

```
3.py - C:/Users/chaht/AppData/Local/Programs/Python/Python313/3.py (3.13.5)
File Edit Format Run Options Window Help

import random
import numpy as np

Q = np.zeros((2, 2))

alpha = 0.1
gamma = 0.9

for _ in range(1000):
    state = random.randint(0, 1)
    action = np.argmax(Q[state])

    reward = 10 if state == action else -5
    next_state = random.randint(0, 1)

    Q[state][action] += alpha * (
        reward + gamma * np.max(Q[next_state]) - Q[state][action]
    )

print("Q-Table:")
print(Q)
```

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

```
>>> ----- RESTART: C:/Users/chaht/AppData/Local/Programs/python/Python313/3.py -----
Q-Table:
[[99.31720095  0.          ]
 [-0.3209      99.32376465]]
>>>
```

Ln: 8 Col: 0

Ln: 1 Col: 0

4. Evaluate the effectiveness of Reinforcement Learning in predictive maintenance compared to traditional rule-based approaches. Discuss risks and reliability concerns in industrial environments.

Explanation

RL predicts machine failures and schedules maintenance before breakdown occurs.

Comparison

Reinforcement Learning	Rule-Based System
Learns from data	Uses fixed rules
Adapts to new failures	Cannot adapt easily
Improves over time	Requires manual updates
Reduces downtime	Less flexible

Risks

- Incorrect learning may cause maintenance delays.
- Requires high-quality sensor data.
- Needs sufficient training before deployment.

Reliability

- Continuous monitoring improves prediction accuracy.
- Prevents unexpected machine failures.

Conclusion: RL is more intelligent and adaptive than traditional rule-based maintenance.

CODE:

```
import random
```

```
import numpy as np
```

```
states=["Healthy","Fault"]
```

```
actions=["Continue","Repair"]
```

```
Q=np.zeros((2,2))
```

```
alpha=0.1
```

```
gamma=0.9
```

```
for i in range(1000):
```

```
    state=random.randint(0,1)
```

```
    action=np.argmax(Q[state])
```

```
    if state==1 and action==1:
```

```
        reward=10
```

```
    elif state==0 and action==0:
```

```
        reward=5
```

```
    else:
```

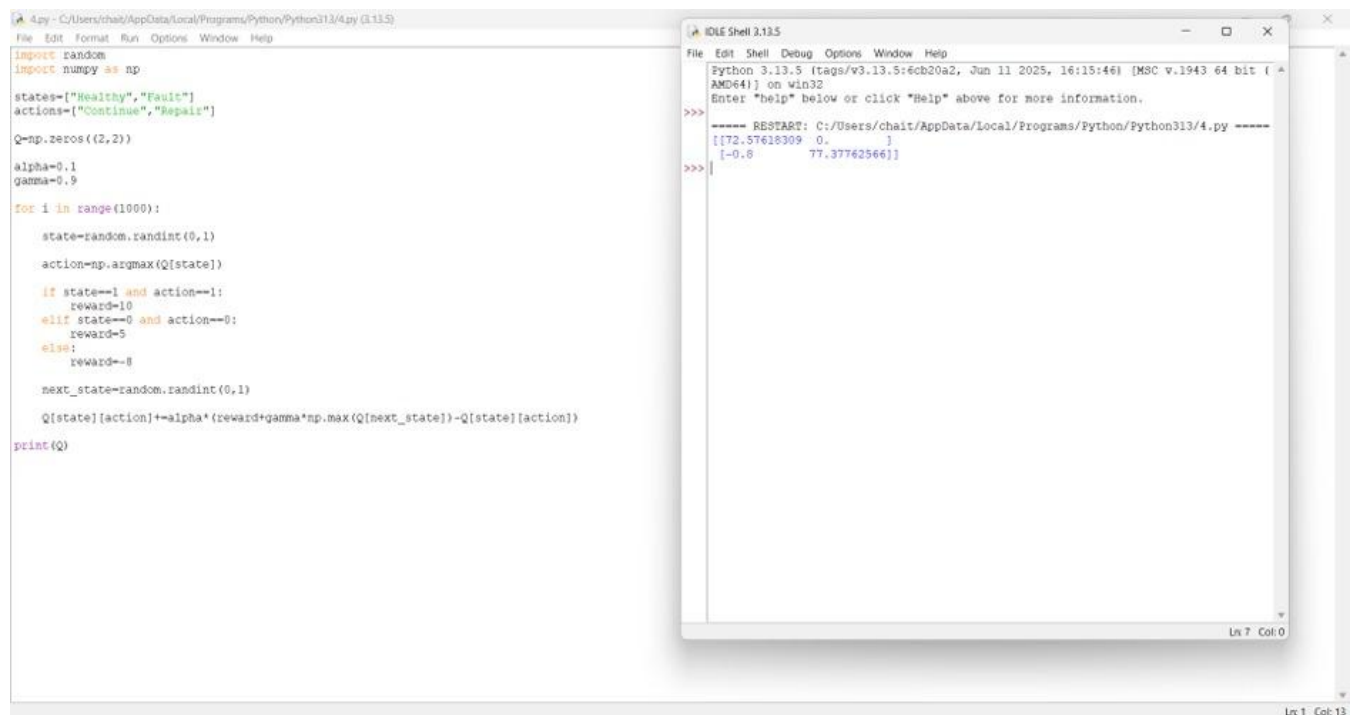
```
        reward=-8
```

```
next_state=random.randint(0,1)
```

```
Q[state][action]+=alpha*(reward+gamma*np.max(Q[next_state])-Q[state][action])
```

```
print(Q)
```

OUTPUT:



The image shows a screenshot of a Python IDE with two windows. The left window displays a script for a Q-learning algorithm. The script imports random and numpy, defines states and actions, initializes a Q-table, and runs a loop for 1000 iterations. The right window shows the output of the script, which is a 2x2 matrix representing the Q-table.

```
4.py - C:/Users/chaht/AppData/Local/Programs/Python/Python313/4.py (3.13.5)
File Edit Format Run Options Window Help

import random
import numpy as np

states=["Healthy","Fault"]
actions=["Continue","Repair"]

Q=np.zeros((2,2))

alpha=0.1
gamma=0.9

for i in range(1000):
    state=random.randint(0,1)
    action=np.argmax(Q[state])

    if state==1 and action==1:
        reward=10
    elif state==0 and action==0:
        reward=5
    else:
        reward=-8

    next_state=random.randint(0,1)

    Q[state][action]+=alpha*(reward+gamma*np.max(Q[next_state])-Q[state][action])

print(Q)
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
----- RESTART: C:/Users/chaht/AppData/Local/Programs/Python/Python313/4.py -----
[[72.57618309  0.]
 [-0.8       77.37762566]]
>>>
```

Ln 7 Col 0

Ln 1 Col 13

5. Analyze how Reinforcement Learning can be used to optimize transportation systems such as bus scheduling or route allocation. Define states, actions, and rewards, and discuss how real-time data improves performance.

Explanation

RL optimizes bus schedules and route allocation based on passenger demand and traffic.

RL Components

- **States:** Traffic level, passenger count, bus location.
- **Actions:** Change route, adjust bus frequency, modify schedule.
- **Rewards:**
 - +10 for on-time arrival
 - +5 for high passenger satisfaction
 - -10 for delays
 - -5 for fuel wastage

Real-Time Data

- Uses GPS data.
- Monitors traffic conditions.
- Adjusts routes dynamically.
- Improves passenger satisfaction.

Conclusion: RL enhances transportation efficiency using real-time information.

CODE:

```
traffic = input("Enter Traffic (High/Low): ").lower()
```

```
passengers = int(input("Enter Number of Passengers: "))
```

```
print("\n----- Current State -----")
```

```
print("Traffic :", traffic)
```

```
print("Passengers :", passengers)
```

```
if traffic == "high" and passengers > 40:
```

```
    action = "Change Route and Add Extra Bus"
```

```
    reward = 25
```

```
elif traffic == "high":
```

```
    action = "Change Route"
```

```
    reward = 15
```

```
elif passengers > 40:
```

```
    action = "Increase Bus Frequency"
```

```
    reward = 20
```

```
else:
```

```
    action = "Continue Current Route"
```

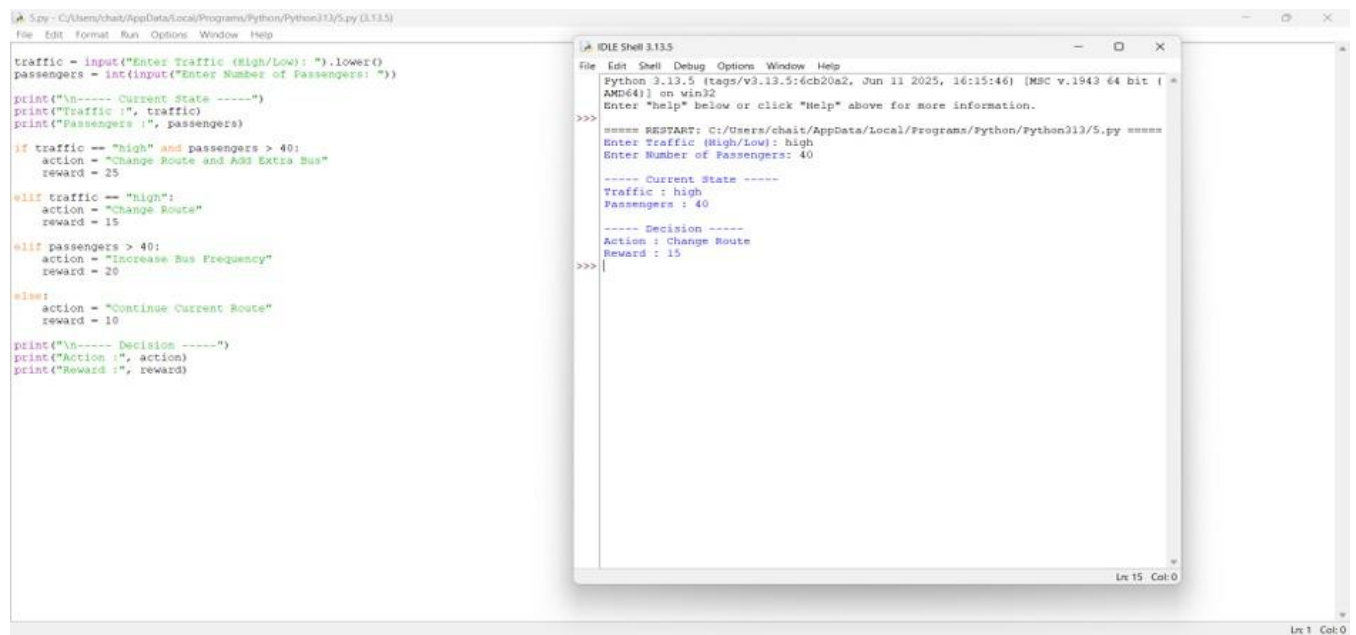
```
    reward = 10
```

```
print("\n----- Decision -----")
```

```
print("Action :", action)
```

```
print("Reward :", reward)
```

OUTPUT:



The image shows a screenshot of a Python IDE with two windows. The main window on the left displays a Python script, and a smaller window on the right shows the output of the script's execution.

Main Window (Python Script):

```
5.py - C:/Users/chaik/AppData/Local/Programs/Python/Python313/5.py (3.13.5)
File Edit Format Run Options Window Help

traffic = input("Enter Traffic (High/Low): ").lower()
passengers = int(input("Enter Number of Passengers: "))

print("\n----- Current State -----")
print("Traffic :", traffic)
print("Passengers :", passengers)

if traffic == "high" and passengers > 40:
    action = "Change Route and Add Extra Bus"
    reward = 25
elif traffic == "high":
    action = "Change Route"
    reward = 15
elif passengers > 40:
    action = "Increase Bus Frequency"
    reward = 20
else:
    action = "Continue Current Route"
    reward = 10

print("\n----- Decision -----")
print("Action :", action)
print("Reward :", reward)
```

Output Window (IDLE Shell 3.13.5):

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/chaik/AppData/Local/Programs/Python/Python313/5.py =====
Enter Traffic (High/Low): high
Enter Number of Passengers: 40

----- Current State -----
Traffic : high
Passengers : 40

----- Decision -----
Action : Change Route
Reward : 15

>>>
```

6. Apply Reinforcement Learning to dynamic pricing in e-commerce. Define the state space, action space, and reward function. Discuss challenges in balancing exploration and exploitation.

Explanation

RL automatically adjusts product prices to maximize sales and profit.

RL Components

- **State Space:** Product demand, competitor price, inventory level, season.
- **Action Space:** Increase price, decrease price, maintain current price.
- **Reward Function:**
 - +10 for higher profit
 - +5 for increased sales
 - -10 for loss of customers
 - -5 for unsold inventory

Exploration vs Exploitation

- **Exploration:** Try new prices.
- **Exploitation:** Use the best-performing prices.

Challenges

- Too much exploration may reduce revenue.
- Too little exploration may miss better pricing strategies.

Conclusion: RL balances pricing decisions to maximize long-term profit.

CODE:

```
price = 1000
```

```
demand = input("Enter Demand (High/Medium/Low): ").lower()
```

```
stock = int(input("Enter Available Stock: "))
```



```
print("\n----- Current State -----")
```

```
print("Current Price :", price)
```

```
print("Demand :", demand)
```

```
print("Stock :", stock)
```

```
if demand == "high" and stock > 20:
```

```
    action = "Increase Price"
```

```
    price += 100
```

```
    reward = 25
```

```
elif demand == "low":
```

```
    action = "Decrease Price"
```

```
    price -= 100
```

```
    reward = 10
```

```
else:
```

```
    action = "Keep Same Price"
```

```
    reward = 15
```

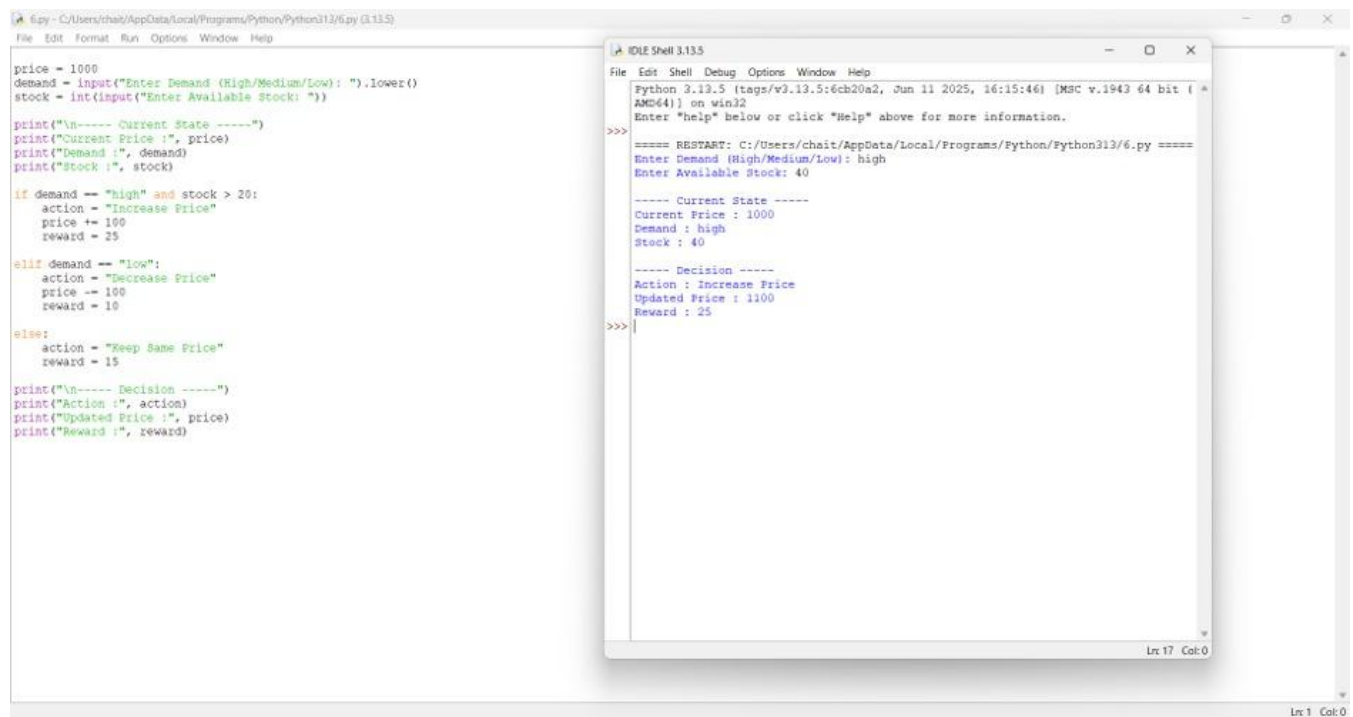
```
print("\n----- Decision -----")
```

```
print("Action :", action)
```

```
print("Updated Price :", price)
```

```
print("Reward :", reward)
```

OUTPUT:



The image shows a screenshot of a Python IDE with two windows. The left window displays the source code for a program, and the right window shows the output of the program.

Source Code (Left Window):

```
price = 1000
demand = input("Enter Demand (High/Medium/Low): ").lower()
stock = int(input("Enter Available Stock: "))

print("\n----- Current State -----")
print("Current Price :", price)
print("Demand :", demand)
print("Stock :", stock)

if demand == "high" and stock > 20:
    action = "Increase Price"
    price += 100
    reward = 25
elif demand == "low":
    action = "Decrease Price"
    price -= 100
    reward = 10
else:
    action = "Keep Same Price"
    reward = 15

print("\n----- Decision -----")
print("Action :", action)
print("Updated Price :", price)
print("Reward :", reward)
```

Output (Right Window):

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
>>> Enter "help" below or click "Help" above for more information.
===== RESTART: C:/Users/chait/AppData/Local/Programs/Python/Python313/6.py =====
Enter Demand (High/Medium/Low): high
Enter Available Stock: 40

----- Current State -----
Current Price : 1000
Demand : high
Stock : 40

----- Decision -----
Action : Increase Price
Updated Price : 1100
Reward : 25
>>>
```

7. Illustrate the use of Reinforcement Learning in smart city waste management systems. Identify the MDP components and discuss how continuous updates improve operational efficiency.

Explanation

RL optimizes waste collection routes and schedules in smart cities.

MDP Components

- **States:** Bin fill level, truck location, traffic conditions.
- **Actions:** Select next bin, change collection route, dispatch another truck.
- **Rewards:**
 - +10 for timely collection
 - +5 for reduced fuel usage
 - -10 for overflowing bins
 - -5 for unnecessary trips

Continuous Updates

- Uses IoT sensor data.
- Learns daily waste generation patterns.
- Optimizes routes continuously.
- Reduces operational costs.

Conclusion: RL improves waste collection efficiency and saves fuel.

CODE:

```
import random
```

```
# Initial waste level (%)
```

```
waste_level = 70
```

```
total_reward = 0
```

```
print("Smart City Waste Management using Reinforcement Learning\n")
```

```
for day in range(10):
```

```
    action = random.choice(["Collect Waste", "Skip"])
```

```
    if action == "Collect Waste":
```

```
        waste_level -= 15
```

```
        reward = 10
```

```
    else:
```

```
        waste_level += 10
```

```
        reward = -5
```

```
    # Keep waste level within 0-100%
```

```
    waste_level = max(0, min(100, waste_level))
```

```
    total_reward += reward
```

```
    print("Day:", day + 1)
```

```
print("Action:", action)
```

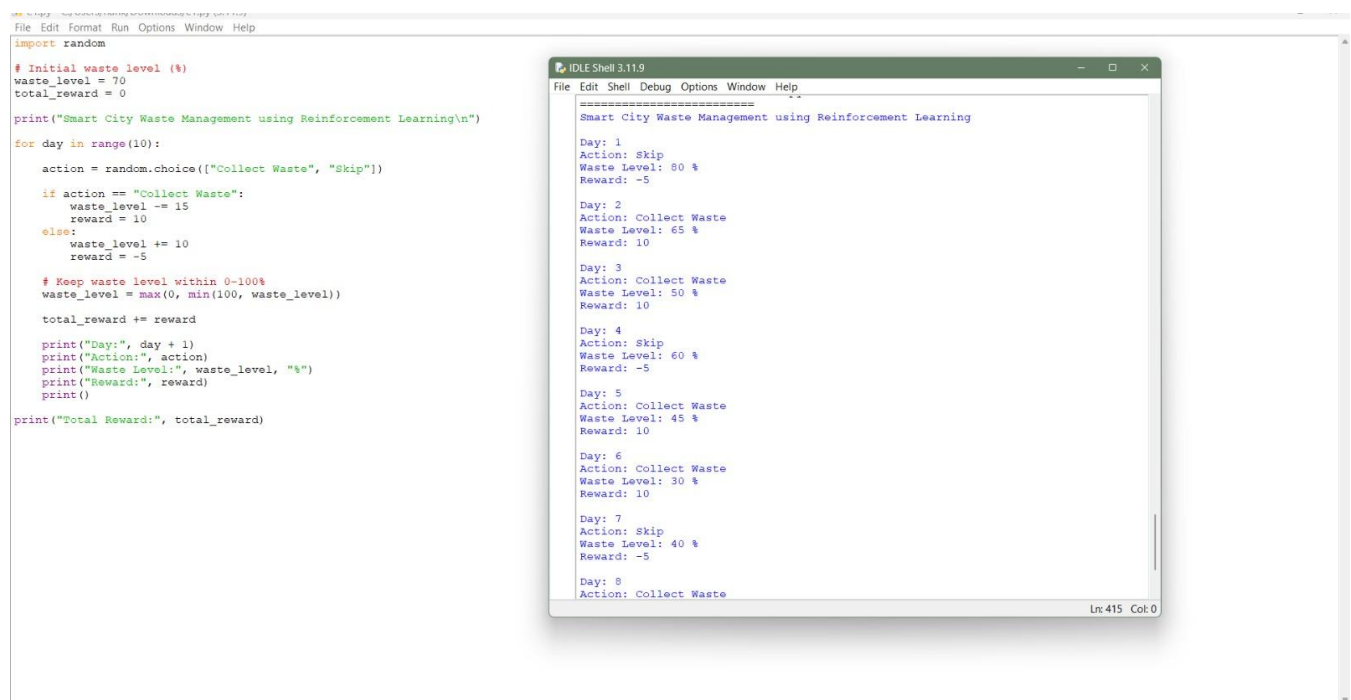
```
print("Waste Level:", waste_level, "%")
```

```
print("Reward:", reward)
```

```
print()
```

```
print("Total Reward:", total_reward)
```

OUTPUT:



The image shows a screenshot of a Python script in an IDE (IDLE 3.11.9) and its output in a shell window. The script simulates a waste management system over 10 days. It starts with a waste level of 70 and a total reward of 0. Each day, it randomly chooses between 'Collect Waste' and 'Skip'. If 'Collect Waste' is chosen, the waste level decreases by 15 and the reward increases by 10. If 'Skip' is chosen, the waste level increases by 10 and the reward decreases by 5. The waste level is kept within 0-100%. The output shows the results for each day, including the action taken, the waste level percentage, and the reward.

```
import random

# Initial waste level (%)
waste_level = 70
total_reward = 0

print("Smart City Waste Management using Reinforcement Learning\n")

for day in range(10):
    action = random.choice(["Collect Waste", "Skip"])

    if action == "Collect Waste":
        waste_level -= 15
        reward = 10
    else:
        waste_level += 10
        reward = -5

    # Keep waste level within 0-100%
    waste_level = max(0, min(100, waste_level))

    total_reward += reward

    print("Day:", day + 1)
    print("Action:", action)
    print("Waste Level:", waste_level, "%")
    print("Reward:", reward)
    print()

print("Total Reward:", total_reward)
```

```
Smart City Waste Management using Reinforcement Learning

Day: 1
Action: Skip
Waste Level: 80 %
Reward: -5

Day: 2
Action: Collect Waste
Waste Level: 65 %
Reward: 10

Day: 3
Action: Collect Waste
Waste Level: 50 %
Reward: 10

Day: 4
Action: Skip
Waste Level: 60 %
Reward: -5

Day: 5
Action: Collect Waste
Waste Level: 45 %
Reward: 10

Day: 6
Action: Collect Waste
Waste Level: 30 %
Reward: 10

Day: 7
Action: Skip
Waste Level: 40 %
Reward: -5

Day: 8
Action: Collect Waste
```

Ln: 415 Col: 0

8. Evaluate how Reinforcement Learning can be used in network bandwidth allocation in telecommunications. Define states, actions, and rewards, and examine how the system adapts to dynamic conditions.

Explanation

RL dynamically allocates internet bandwidth based on network traffic.

RL Components

- **States:** Network load, bandwidth usage, user demand.
- **Actions:** Increase bandwidth, decrease bandwidth, prioritize users.
- **Rewards:**
 - +10 for smooth communication
 - +5 for high throughput
 - -10 for congestion
 - -5 for packet loss

Dynamic Adaptation

- Monitors network traffic continuously.
- Adjusts bandwidth allocation automatically.
- Maintains better Quality of Service (QoS).

Conclusion: RL improves network performance under changing conditions.

CODE:

```
import random
```

```
bandwidth = 60
```

```
total_reward = 0
```

```
print("Network Bandwidth Allocation using Reinforcement Learning\n")
```

```
for step in range(10):
```

```
    action = random.choice(["Increase", "Decrease"])
```

```
    if action == "Increase":
```

```
        if bandwidth < 80:
```

```
            bandwidth += 10
```

```
            reward = 10    # Good decision
```

```
        else:
```

```
            reward = -5    # Network is already busy
```

```
    else:
```

```
        if bandwidth > 20:
```

```
            bandwidth -= 10
```

```
            reward = 5    # Saves bandwidth
```

```
        else:
```

```
            reward = -5    # Too little bandwidth allocated
```

```
bandwidth = max(0, min(100, bandwidth))
```

```
total_reward += reward
```

```
print("Step:", step + 1)
```

```
print("Action:", action)
```

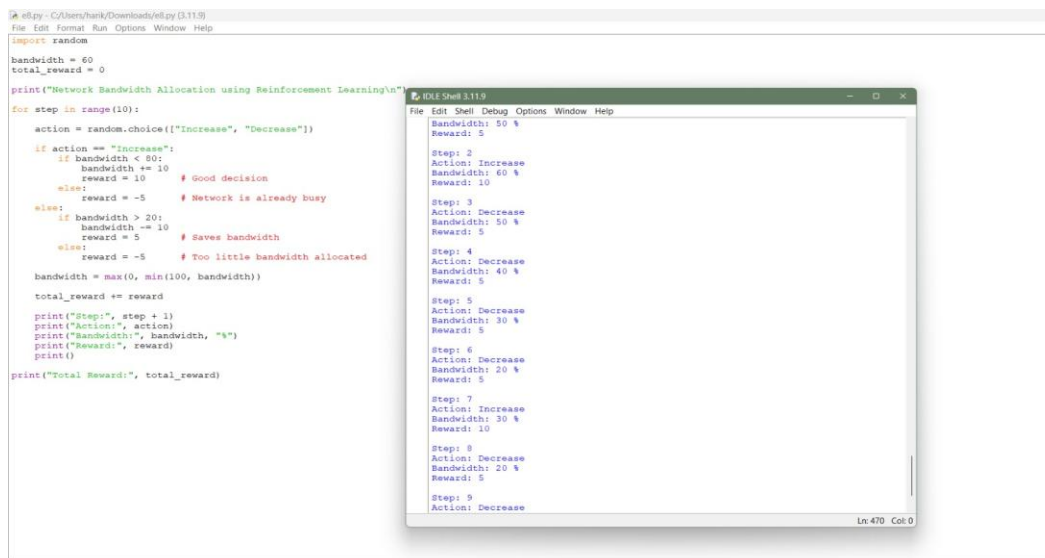
```
print("Bandwidth:", bandwidth, "%")
```

```
print("Reward:", reward)
```

```
print()
```

```
print("Total Reward:", total_reward)
```

OUTPUT:



The screenshot shows a Jupyter Notebook on the left and its output console on the right. The notebook contains a Python script for a reinforcement learning simulation. The script initializes bandwidth at 50 and total_reward at 0. It runs for 10 steps, choosing between 'Increase' and 'Decrease' actions based on the current bandwidth. The output console displays the results of these actions, showing how the bandwidth fluctuates and the total reward is calculated.

```
bandwidth = 50
total_reward = 0

print("Network Bandwidth Allocation using Reinforcement Learning\n")

for step in range(10):
    action = random.choice(["Increase", "Decrease"])

    if action == "Increase":
        if bandwidth < 80:
            bandwidth += 10
            reward = 10 # Good decision
        else:
            reward = -5 # Network is already busy
    else:
        if bandwidth > 20:
            bandwidth -= 10
            reward = 5 # Saves bandwidth
        else:
            reward = -5 # Too little bandwidth allocated

    bandwidth = max(0, min(100, bandwidth))
    total_reward += reward

    print("Step:", step + 1)
    print("Action:", action)
    print("Bandwidth:", bandwidth, "%")
    print("Reward:", reward)
    print()

print("Total Reward:", total_reward)
```

Output:

```
Bandwidth: 50 %
Reward: 5

Step: 2
Action: Increase
Bandwidth: 60 %
Reward: 10

Step: 3
Action: Decrease
Bandwidth: 50 %
Reward: 5

Step: 4
Action: Decrease
Bandwidth: 40 %
Reward: 5

Step: 5
Action: Decrease
Bandwidth: 30 %
Reward: 5

Step: 6
Action: Decrease
Bandwidth: 20 %
Reward: 5

Step: 7
Action: Increase
Bandwidth: 30 %
Reward: 10

Step: 8
Action: Decrease
Bandwidth: 20 %
Reward: 5

Step: 9
Action: Decrease
```


9. Analyze the application of Reinforcement Learning in portfolio management in finance. Discuss advantages over traditional models and challenges such as risk and delayed rewards.

Explanation

RL helps investors decide how to allocate money among different financial assets.

Advantages

- Learns from market changes.
- Optimizes long-term returns.
- Adapts investment strategies automatically.

Challenges

- **Risk:** Market volatility may reduce returns.
- **Delayed Rewards:** Investment gains are realized over time.
- Requires large historical datasets.

RL Components

- **States:** Stock prices, market trends, portfolio value.
- **Actions:** Buy, sell, hold assets.
- **Rewards:** Portfolio profit after considering risk.

Conclusion: RL provides adaptive investment strategies but must manage uncertainty and delayed rewards.

CODE:

```
import random
```

```
balance = 1000
```

```
for i in range(5):
```

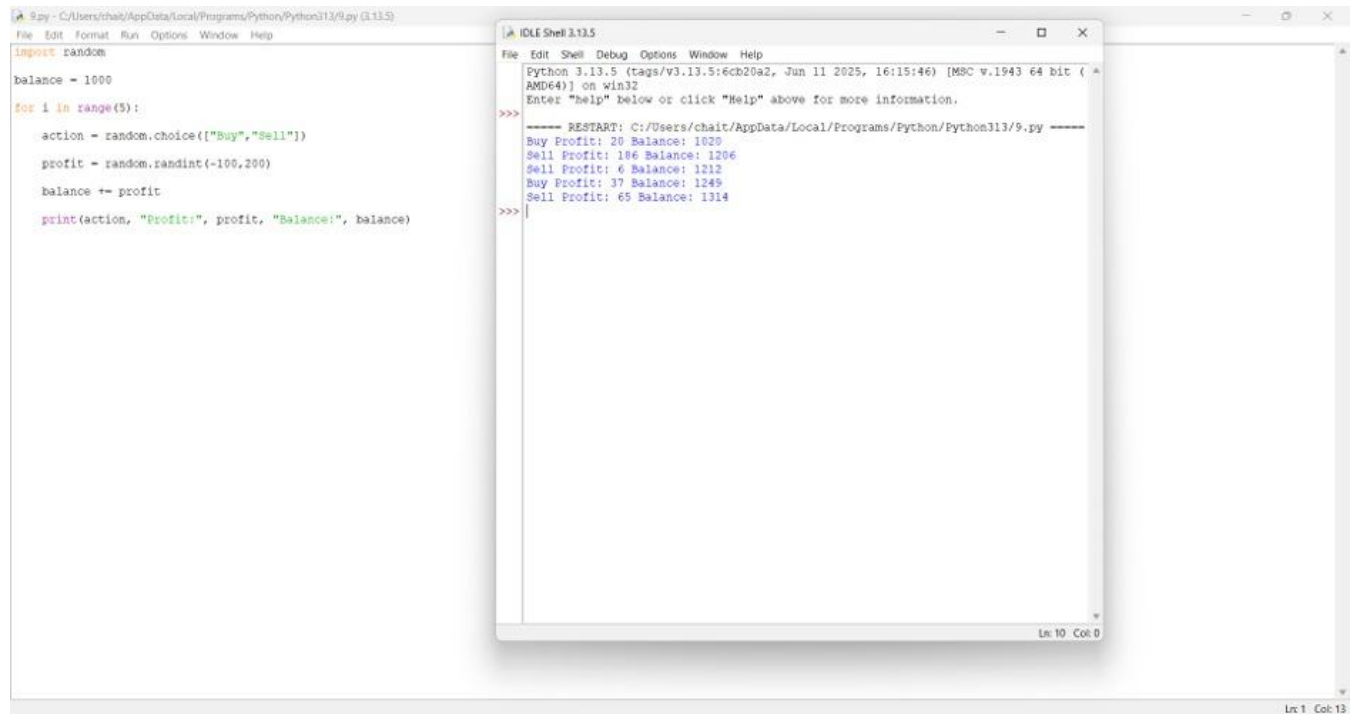
```
    action = random.choice(["Buy","Sell"])
```

```
    profit = random.randint(-100,200)
```

```
    balance += profit
```

```
    print(action, "Profit:", profit, "Balance:", balance)
```

OUTPUT:



The screenshot shows a Python IDE with two windows. The left window displays the code from the previous blocks. The right window, titled 'IDLE Shell 3.13.5', shows the output of the program. The output indicates a restart of the script and lists five transactions with their respective actions, profits, and balances.

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
----- RESTART: C:/Users/chaik/AppData/Local/Programs/Python/Python313/9.py -----
Buy Profit: 20 Balance: 1020
Sell Profit: 186 Balance: 1206
Sell Profit: 6 Balance: 1212
Buy Profit: 37 Balance: 1249
Sell Profit: 65 Balance: 1314
>>>
```

Ln 1 Col 13

10. Justify the use of Reinforcement Learning in renewable energy management systems. Define the RL framework components and discuss the impact of environmental uncertainty on decision-making.

Explanation

RL manages renewable energy resources such as solar and wind power by balancing energy generation, storage, and consumption.

RL Components

- **States:** Energy demand, battery level, weather conditions, power generation.
- **Actions:** Store energy, supply electricity, purchase backup power.
- **Rewards:**
 - +10 for efficient energy use
 - +5 for reduced operating cost
 - -10 for power shortages
 - -5 for energy wastage

Environmental Uncertainty

- Weather conditions change frequently.
- RL learns to handle uncertain solar and wind generation.
- Improves battery usage and power distribution.

Conclusion: RL enables efficient renewable energy management despite changing environmental conditions.

CODE:

```
import random
```

```
battery = 50
```

```
for i in range(5):

    action = random.choice(["Store","Supply"])

    if action == "Store":

        battery += 10

    else:

        battery -= 10

    reward = random.randint(1,10)

    print(action, "Battery:", battery, "Reward:", reward)
```

OUTPUT:

The image shows a screenshot of a Python IDE with two windows. The main window displays a Python script for a battery management simulation. The script initializes a battery at 50 and runs a loop for 5 iterations. In each iteration, it randomly chooses between 'Store' and 'Supply' actions. 'Store' increases the battery level by 10, while 'Supply' decreases it by 10. A random reward between 1 and 10 is generated for each action, and the current state is printed. An 'IDLE Shell' window is open in the foreground, showing the execution output. It displays a restart message followed by five lines of simulation results, each showing the action taken, the resulting battery level, and the reward received.

```
10.py - C:/Users/chaht/AppData/Local/Programs/Python/Python313/10.py (3.13.5)
File Edit Format Run Options Window Help

import random

battery = 50

for i in range(5):

    action = random.choice(["Store","Supply"])

    if action == "Store":
        battery += 10
    else:
        battery -= 10

    reward = random.randint(1,10)

    print(action, "Battery:", battery, "Reward:", reward)

IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
----- RESTART: C:/Users/chaht/AppData/Local/Programs/Python/Python313/10.py -----
Supply Battery: 40 Reward: 7
Supply Battery: 30 Reward: 8
Supply Battery: 20 Reward: 10
Store Battery: 30 Reward: 9
Store Battery: 40 Reward: 5
>>>
```

Code of Conduct

Name / Reg No: _____

I certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____